Project Title

Resume job Matching & Talent markatplace engine

Project Report submitted for the partial fulfilment of grade for the course

Data Structures and Algorithms -3 (25CS2103E)

in

A.Y. 2026-2027

Submitted by
R. Rohit – 2520080037
G.V. Karthik Reddy – 2520080006

Team Number: 16

Section: 12

Under the guidance of

Dr. Chandusha Kanda
CS&IT / AI&DS



Hyderabad-500090, Telangana, India.

K L (Deemed to be) University

DEPARTMENT OF COMPUTER SCIENCE ENGINEERING / COMPUTER SCIENCE AND INFORMATION TECHNOLOGY/ARTIFICIAL INTELLIGENCE & DATA SCIENCE



DECLARATION

We hereby declare that the mini-project report entitled **"Résumé–Job Matching & Talent Marketplace Engine"** is a record of the work carried out by us in partial fulfilment of the requirements for the course **Data Structures and Algorithms – 3 (25CS2103E)** at K L (Deemed to be) University during the academic year 2026–2027.

The project focuses on applying fundamental Data Structures and Algorithms to the problem of connecting job seekers with suitable employment opportunities and helping recruiters identify relevant candidates. The proposed system incorporates skill-based matching, searching, sorting, ranking, and recommendation techniques, along with skill-gap analysis and career-roadmap generation.

We declare that this report has been prepared for academic purposes and that the work presented is our own, except where external resources and references have been acknowledged. We understand that all project results and implementation claims must accurately reflect the work completed by our team.

Student 1: R. Rohit – 2520080037

Student 2: G V Karthik Reddy – 2520080006

K L (Deemed to be) University

DEPARTMENT OF COMPUTER SCIENCE ENGINEERING / COMPUTER SCIENCE AND INFORMATION TECHNOLOGY



CERTIFICATE

This is to certify that the mini-project report entitled **"Résumé–Job Matching & Talent Marketplace Engine"** is a bona fide record of the work carried out by R. Rohit - 2520080037, G.V.Karthik Reddy - 2520080006 in partial fulfilment of the requirements for the course **Data Structures and Algorithms – 3 (25CS2103E)** at K L (Deemed to be) University during the academic year 2026–2027.

The project demonstrates the application of Data Structures and Algorithms to an online recruitment and talent-matching problem. The report has been prepared under the guidance and supervision of the undersigned.

Project Guide: Dr. Chandusha Kanda

Department: CS&IT / AI&DS

Signature of the Guide: _____

Signature of the Head of the Department: _____

Signature of the Guide **Signature of the HOD**

ACKNOWLEDGEMENT

The success in this project would not have been possible but for the timely help and guidance rendered by many people. Our wish to express my sincere thanks to all those who has assisted us in one way or the other for the completion of my project.

Our greatest appreciation to my guide, Dr. Chandusha Kanda, Assistant Professor, CS&IT which cannot be expressed in words for his/her tremendous support, encouragement and guidance for this project.

We express our gratitude to **Dr. Khaja Shareef (CSIT)/ Dr. Sreenath Itikela (AI&DS)**, Head of the *Department for Computer Science and Information Technology/Department of Artificial Intelligence and Data Science* for providing us with adequate facilities, ways and means by which we are able to complete this project-based Lab.

We thank all the members of teaching and non-teaching staff members, and also who have assisted me directly or indirectly for successful completion of this project. Finally, I sincerely thank my parents, friends and classmates for their kind help and cooperation during my work.

R. Rohit – 2520080037

G.V.Karthik Reddy - 2520080006

TABLE OF CONTENTS

S.No.	Contents	Page no.
1.	Abstract	8
2.	Introduction	9
3.	Literature survey	10
4.	Problem Statement and Objectives	12
5.	Hardware and Software requirements	14
6.	Implementation	16
7.	Results & Discussion	24
8.	Conclusion & Future Scope	29
9.	References	32

1.ABSTRACT

The **Résumé–Job Matching & Talent Marketplace Engine** is a web-based application designed to improve the process of connecting job seekers with suitable employment opportunities and enabling recruiters to identify relevant candidates. Conventional job portals often return large numbers of search results, making it difficult for users to identify opportunities or candidates that closely match their requirements.

The proposed system provides separate functionalities for candidates and recruiters. Candidates can create professional profiles, maintain their skills and qualifications, explore job opportunities, and receive ranked job recommendations. Recruiters can publish job openings, specify required skills, and view candidates ranked according to their suitability for a position. A skill-gap analysis module identifies the differences between a candidate's existing skills and a target job's requirements. The Reverse Resume component extends this functionality by outlining skills that a candidate may need to acquire to work toward a desired role.

The project emphasizes the practical implementation of Data Structures and Algorithms, including hashing for efficient skill lookup, tries for search autocomplete, searching and sorting algorithms for information retrieval and ordering, priority queues for selecting top recommendations, and graph representations for modelling relationships among candidates, skills, and jobs. A weighted ranking mechanism can combine skill, education, experience, and project relevance into a compatibility score.

The system aims to demonstrate how algorithmic techniques can improve the efficiency, organization, and usefulness of recruitment-related searches while providing candidates with actionable career-development guidance.

Team Members:

1. R. Rohit – 2520080037
2. G.V.Karthik Reddy - 2520080006

2. INTRODUCTION

The rapid growth of online recruitment platforms has transformed the way job seekers discover employment opportunities and recruiters identify suitable candidates. Digital platforms enable users to create professional profiles, upload résumés, search for job openings, and connect with potential employers. However, the increasing volume of job listings and candidate profiles creates challenges in identifying relevant opportunities efficiently. Candidates may find it difficult to determine which jobs best match their skills and qualifications, while recruiters may spend considerable time identifying applicants who satisfy specific job requirements.

The **Résumé–Job Matching & Talent Marketplace Engine** is a web-based application designed to address these challenges through automated skill-based matching, searching, ranking, and recommendation. The system allows candidates to maintain professional profiles, explore job opportunities, and receive ranked recommendations based on their skills and qualifications. Recruiters can publish job openings, specify required skills, and discover candidates ranked according to their suitability for a particular position. Additionally, the Reverse Resume feature aims to identify skill gaps and provide career-development guidance to help candidates work toward their desired roles.

A key aspect of this project is the practical application of **Data Structures and Algorithms (DSA)** to improve the efficiency of recruitment-related operations. Hashing can support fast skill lookup, tries can enable prefix-based autocomplete, searching and sorting algorithms can retrieve and organize relevant results, and heaps or priority queues can efficiently identify top-ranked recommendations. Graphs can represent relationships among candidates, skills, and jobs, while ranking algorithms can calculate compatibility scores using relevant candidate and job attributes. These techniques make the project more than a conventional data-management application by emphasizing algorithmic problem-solving and efficient information retrieval.

The primary objective of the project is to develop a functional talent marketplace that connects candidates with suitable employment opportunities while helping recruiters identify relevant talent. It also aims to demonstrate the implementation, analysis, and practical usefulness of DSA concepts within a real-world application scenario.

The remainder of this report presents a literature survey of existing recruitment platforms, the problem statement and project objectives, hardware and software requirements, the implementation of the proposed system, results and discussion, and the conclusion with possible future enhancements.

3. LITERATURE SURVEY

3.1 LinkedIn Jobs

LinkedIn Jobs is an online recruitment service that connects job seekers with employers through professional profiles, job listings, and job recommendations. Candidates can maintain information about their skills and experience, while recruiters can discover potential applicants. This platform is relevant to the proposed project because it illustrates the importance of profile-based job discovery and matching. The proposed system focuses specifically on demonstrating the underlying searching, ranking, and recommendation techniques using Data Structures and Algorithms.

Website: https://www.linkedin.com/jobs/

3.2 Naukri.com

Naukri.com is an Indian recruitment platform that provides job listings, résumé management, and recruitment services. Candidates can create professional profiles and search for opportunities, while recruiters can identify candidates based on job requirements and qualifications. The platform is particularly relevant to the proposed system because of its focus on the Indian employment market. The proposed project builds on this general recruitment model by incorporating explicit DSA-based matching and recommendation mechanisms.

Website: https://www.naukri.com/

3.3 Internshala

Internshala provides internship and entry-level job opportunities, making it particularly relevant to students and recent graduates. Users can search for opportunities and apply according to their qualifications and interests. This student-oriented approach aligns with the proposed project's goal of helping candidates understand the skills required for target roles. The Reverse Resume component extends this idea by identifying missing skills and suggesting a structured path toward a desired career.

Website: https://internshala.com/

3.4 Wellfound

Wellfound connects job seekers with companies, particularly within the startup ecosystem. Its recruitment model is relevant to candidates looking for roles in technology-oriented organizations. It demonstrates how a talent marketplace can connect professional profiles with company opportunities. The proposed project adopts a similar candidate–recruiter interaction model while concentrating on algorithmic matching and educational career guidance.

Website: https://wellfound.com/

4. Problem Statement and Objectives

Problem Statement:

To develop a web-based résumé–job matching and talent marketplace system that uses Data Structures and Algorithms to efficiently identify, rank, and recommend suitable jobs for candidates and suitable candidates for recruiters.

Problem Description:

Job seekers may struggle to identify relevant opportunities among numerous listings, while recruiters may find it difficult to identify candidates whose qualifications match specific job requirements. Conventional keyword-based searching may not adequately communicate the degree of compatibility between a candidate's skills and a job's requirements.

An efficient matching system is therefore required to organize candidate and job information, calculate compatibility scores, rank results, and identify skill gaps that candidates can address to improve their suitability for desired roles.

Objectives:

The primary objectives of the project are:

1. Candidate management: Provide facilities for candidates to create and maintain profiles containing skills, education, projects, and experience.

2. Recruiter management: Enable recruiters to publish job openings with descriptions, qualifications, and required skills.

3. Efficient searching: Implement appropriate searching methods to retrieve jobs and candidates based on user queries and requirements.

4. Skill indexing using hashing: Organize skill information for efficient lookup and retrieval.

5. Autocomplete using tries: Support prefix-based suggestions for skills and job titles.

6. Sorting and ranking: Rank jobs and candidates according to calculated compatibility scores.

7. Priority-based recommendations: Use a heap or priority queue to retrieve the highest-ranked jobs or candidates efficiently.

8. Graph-based relationships: Represent relationships among candidates, skills, jobs, and companies to support related-job or similar-candidate discovery.

9. Skill-gap analysis: Compare a candidate's current skills with the requirements of a selected job and identify missing skills.

10. Reverse Resume roadmap: Help candidates plan the skills and projects needed to work toward a target role.

11. Database integration: Store and retrieve application information using a database such as Supabase, where configured.

12. Algorithmic evaluation: Explain the time and space complexity of the major DSA operations and assess their suitability for the application.

5. HARDWARE AND SOFTWARE REQUIREMENTS

5.1 Hardware Requirements

The following configuration is suitable for developing and demonstrating the project. Actual requirements depend on the size of the dataset and the development environment.

Component	Minimum / Recommended Requirement
Processor	Intel Core i3 or equivalent
RAM	8 GB recommended
Storage	10 GB of available space for source code, dependencies, and project files
Display	Standard laptop or desktop display
Network	Internet connection for Supabase, package installation, and online deployment

5.2 Software Requirements

Software / Technology	Purpose
macOS, Windows, or Linux	Development operating system
Visual Studio Code	Source-code editing and debugging
HTML5	Structuring web pages, where applicable
CSS3	Styling and responsive layouts
JavaScript / TypeScript	Frontend logic and algorithm implementation, depending on the project configuration
Node.js	JavaScript runtime for the server and development tools

Express.js	Backend API framework, if used in the implementation
Next.js or Vite	Frontend application framework, depending on the existing project
Supabase	Hosted PostgreSQL database and authentication services, if configured
Git and GitHub	Version control and source-code hosting
npm	Dependency installation and script execution
Google Chrome or another modern browser	Application testing and demonstration

6. IMPLEMENTATION

The implementation section describes the major functional modules of the system, the data structures used, and the algorithms that support matching and recommendations. The report template requests code for three to four modules.

The following four modules are suitable for the project. Use the code as report material only if it matches your actual implementation; otherwise, replace it with the corresponding code from your project.

6.1 System Architecture

The proposed system follows a layered architecture.

Presentation Layer

Candidate Dashboard · Recruiter Dashboard · Job Search · Career Roadmap

Application and API Layer

Authentication · Profile Management · Job Management · API Validation

DSA Processing Layer

Hashing · Trie · Searching · Sorting · Priority Queue · Graph · Ranking

Data Layer

Supabase / PostgreSQL · Candidate Profiles · Jobs · Skills · Applications

Figure 6.1: Proposed system architecture. Adjust the diagram if your actual implementation uses a different architecture or database.

The presentation layer provides interfaces for candidates and recruiters. The application layer processes requests and coordinates the relevant operations. The DSA processing layer performs skill matching, searching, sorting, ranking, and recommendation tasks. The data layer stores persistent information about users, jobs, skills, and applications.

6.2 Module 1: Candidate and Recruiter Management

Purpose: This module manages user profiles and job-related information.

The candidate workflow includes creating a profile, adding skills and qualifications, and exploring job opportunities. The recruiter workflow includes maintaining company information, creating job listings, and specifying the skills required for each position.

The module can use database tables such as profiles, companies, jobs, skills, and candidate_skills. The actual table names should correspond to the schema created in Supabase.

Processing flow:

1. The user submits registration or profile information.
2. The system validates the submitted data.
3. Authentication and authorization are checked.
4. Valid profile or job information is stored in the database.
5. The application retrieves the information when the relevant dashboard or search page is opened.

DSA relevance: Hash-based lookup structures can index candidate identifiers, job identifiers, or skill names to support efficient retrieval. Database indexing and in-memory hashing are distinct mechanisms and should be described separately in the report.

6.3 Module 2: Skill-Based Matching Engine

Purpose: The matching engine compares candidate skills with the skills required by a job and calculates a compatibility score.

Let:

- CC represent the set of skills possessed by a candidate.
- JJ represent the set of required skills for a job.
- $C \cap J$C∩J represent the skills shared by both sets.

A simple matching score is:

$\text{Skill Match} = \frac{|C \cap J|}{|J|} \times 100$Skill Match=|J||C∩J| ×100

For example, suppose a job requires Python, SQL, React, and Node.js, while the candidate has Python, SQL, and React.

The candidate possesses three of the four required skills.

$\text{Skill Match} = \frac{3}{4} \times 100 = 75\%$Skill Match=43 ×100=75%

The system can use this score to rank jobs for candidates or candidates for recruiters. More advanced ranking can incorporate education, experience, projects, and certifications, provided that these attributes are available and implemented.

Illustrative JavaScript code:

```
function calculateSkillMatch(candidateSkills, jobSkills) {
  const candidateSet = new Set(
    candidateSkills.map(skill => skill.toLowerCase())
  );

  const requiredSkills = [
    ...new Set(jobSkills.map(skill => skill.toLowerCase()))
  ];

  if (requiredSkills.length === 0) {
    return 0;
  }

  const matchedSkills = requiredSkills.filter(
    skill => candidateSet.has(skill)
  );

  return Number(
    ((matchedSkills.length / requiredSkills.length) * 100).toFixed(2)
  );
}

const candidate = ["Python", "SQL", "React"];
const required = ["Python", "SQL", "React", "Node.js"];

console.log(calculateSkillMatch(candidate, required));
// 75
```

Complexity analysis: Building the candidate set takes expected $O(|C|)$O(|C|) time, and checking the required skills takes expected $O(|J|)$O(|J|) time. The overall expected time complexity is $O(|C|+|J|)$O(|C|+|J|), assuming average constant-time set operations.

This is a skill-overlap score, not a complete assessment of professional suitability.

6.4 Module 3: Trie-Based Search Autocomplete

Purpose: A trie is a tree-based data structure that stores strings character by character. It can generate suggestions when a user types a partial skill or job title.

For example, entering Py could return Python and PyTorch if those terms exist in the index.

Illustrative JavaScript code:class TrieNode {
```
  constructor() {
    this.children = new Map();
    this.isEnd = false;
  }
}

class Trie {
  constructor() {
    this.root = new TrieNode();
  }

  insert(word) {
    let node = this.root;

    for (const char of word.toLowerCase()) {
      if (!node.children.has(char)) {
        node.children.set(char, new TrieNode());
      }

      node = node.children.get(char);
    }

    node.isEnd = true;
  }

  searchPrefix(prefix) {
    let node = this.root;

    for (const char of prefix.toLowerCase()) {
      if (!node.children.has(char)) return [];

      node = node.children.get(char);
    }

    const results = [];

    const collect = (current, word) => {
      if (current.isEnd) results.push(word);

      for (const [char, child] of current.children) {
        collect(child, word + char);
```

```
    }
  };

  collect(node, prefix.toLowerCase());
  return results;
 }
}

const trie = new Trie();

["Python", "PyTorch", "Java", "JavaScript"].forEach(
 skill => trie.insert(skill)
);

console.log(trie.searchPrefix("py"));
// ["python", "pytorch"]
```

Complexity analysis:

- Inserting a word of length L: $O(L)$O(L) expected time with hash-map-backed child lookup.
- Finding the node for a prefix of length P: $O(P)$O(P) expected time.
- Collecting suggestions additionally depends on the number of nodes explored and the number of results returned.

The trie improves prefix lookup by avoiding a complete scan of every indexed term. The example returns all matching suggestions; a production implementation can limit the result count.

6.5 Module 4: Ranking and Top-K Recommendations

Purpose: This module orders matching jobs or candidates according to their compatibility scores.

For example, the engine might calculate scores of 92%, 85%, 78%, and 64%. The ranking module arranges the results from highest to lowest score.

A weighted ranking formula may be used when the necessary profile data is available:

$$Score = 0.50S + 0.15E + 0.15X + 0.10P + 0.10C$$\begin{aligned} \text{Score} = {} & 0.50S + 0.15E + 0.15X \\ & + 0.10P + 0.10C \end{aligned}Score=0.50S+0.15E+0.15X+0.10P+0.10C

Where:

- SSS = skill match percentage.
- EEE = education match percentage.
- XXX = experience match percentage.
- PPP = project relevance percentage.
- CCC = certification match percentage.

Each component is normalized to a value from 0 to 100.

Illustrative JavaScript code:

```
function calculateOverallScore(candidate) {
  return (
    0.50 * candidate.skillMatch +
    0.15 * candidate.educationMatch +
    0.15 * candidate.experienceMatch +
    0.10 * candidate.projectMatch +
    0.10 * candidate.certificationMatch
  );
}

function rankCandidates(candidates) {
  return candidates
    .map(candidate => ({
      ...candidate,
      score: calculateOverallScore(candidate)
    }))
    .sort((a, b) => b.score - a.score);
}
```

Complexity analysis: Calculating the score for each candidate takes $O(n)O(n)$O(n) time. Sorting nnn candidates using the language's comparison sort generally takes $O(nlog⁡n)O(n|log n)$O(nlogn) time.

A heap or priority queue can be used when only the top kkk recommendations are needed. A min-heap of size kkk can retain the best kkk candidates in $O(nlog⁡k)O(n|log k)$O(nlogk) time, with $O(k)O(k)$O(k) additional heap storage, assuming $k≤nk|leq nk≤n$. This is particularly useful when the candidate or job collection is large.

6.6 Additional DSA Components

The following components can be included in the implementation discussion if they exist in your source code.

DSA concept	Application in the project	Expected complexity
Hashing	Skill and identifier lookup	Expected $O(1)O(1)$O(1) lookup
Linear search	Searching an unsorted collection	$O(n)O(n)$O(n)
Binary search	Searching a sorted collection	$O(\log n)O(\log n)$O(logn)
Sorting	Ordering jobs or candidates by score	Typically $O(n\log n)O(n\log n)$O(nlogn)
Trie	Prefix-based autocomplete	$O(P)O(P)$O(P) to reach a prefix node, plus result traversal
Priority queue	Retrieving top recommendations	$O(\log k)O(\log k)$O(logk) per heap update for a heap of size kkk
Graph	Representing candidate–skill–job relationships	BFS/DFS: $O(V+E)O(V+E)$O(V+E) with adjacency lists
Ranking formula	Calculating compatibility scores	$O(1)O(1)$O(1) per candidate when component scores are already available

Here, nnn is the collection size, PPP is the prefix length, VVV is the number of graph vertices, and EEE is the number of graph edges.

Important distinction: These are the complexities of the described algorithmic operations. They are not measured performance results for your deployed application.

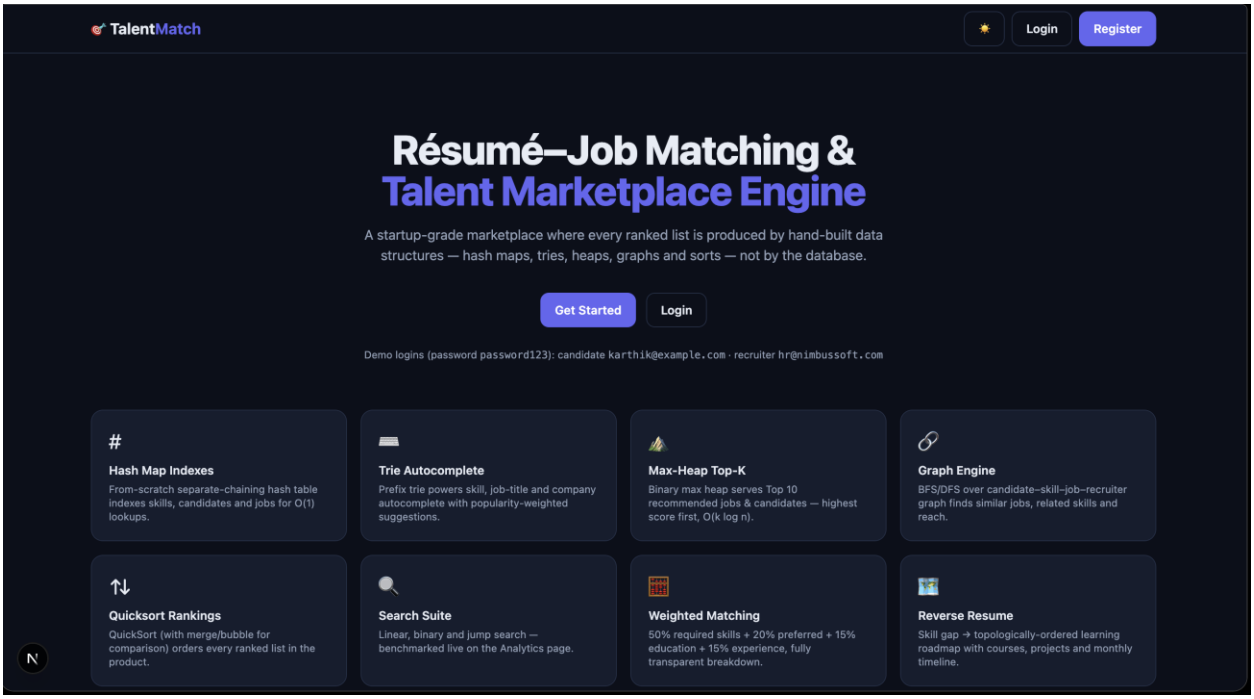
7. RESULTS & DISCUSSION

This section should document what happened when you ran the application. The template specifically asks for application screenshots and a brief explanation of each.

Do not create artificial screenshots or claim successful tests that you have not performed. Capture the actual application running locally or after deployment.

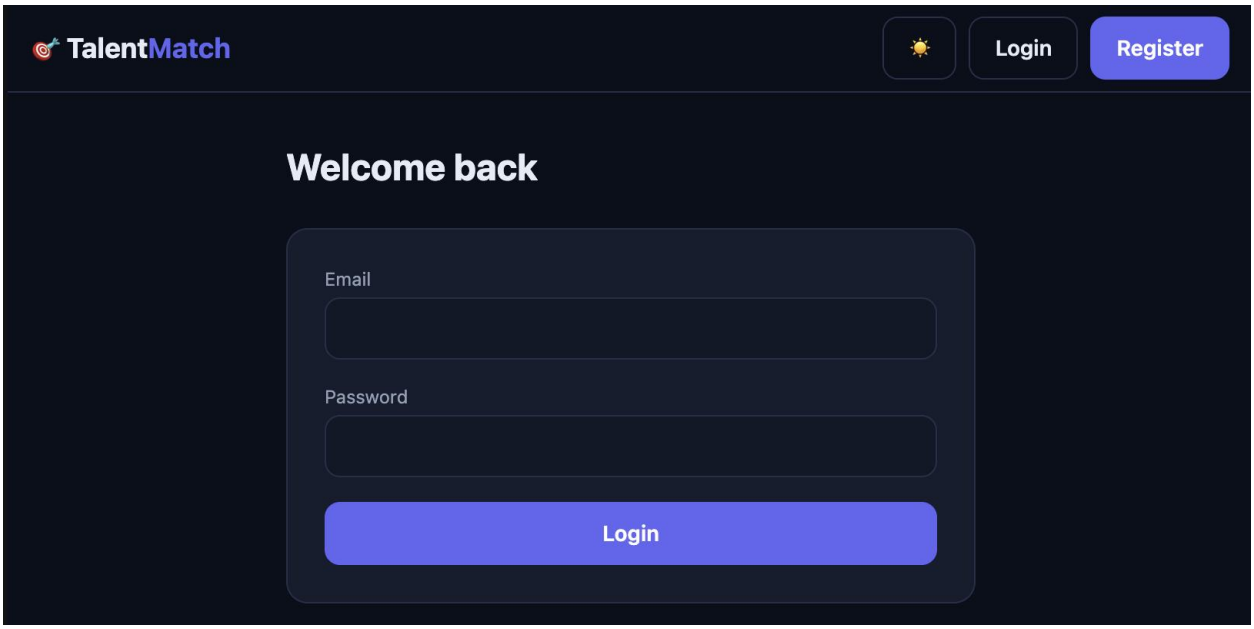
7.1 Application Home Page

Description: The home page introduces the Résumé–Job Matching & Talent Marketplace Engine and provides navigation to the candidate and recruiter workflows. It presents the purpose of the platform and the main services available to users.



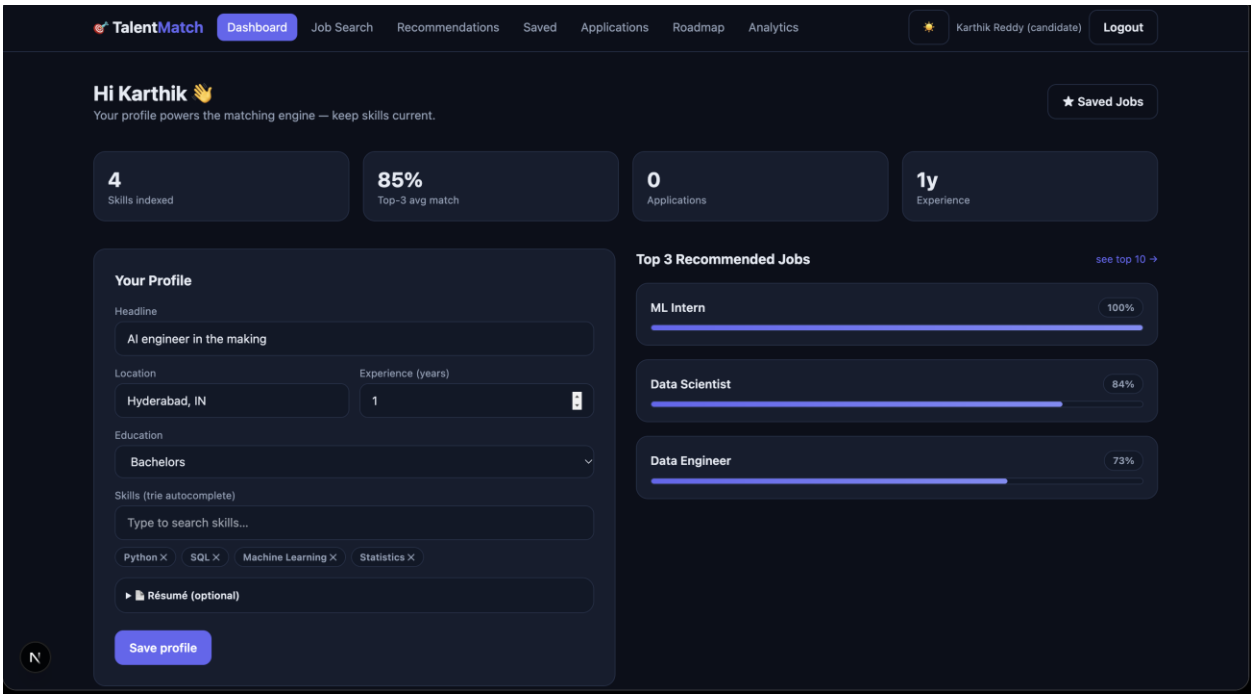
7.2 Candidate Registration and Login

Description: The authentication interface allows candidates to register or sign in to access their profiles and job-related features. Authentication and session persistence should be verified using the configured authentication provider before describing them as operational.



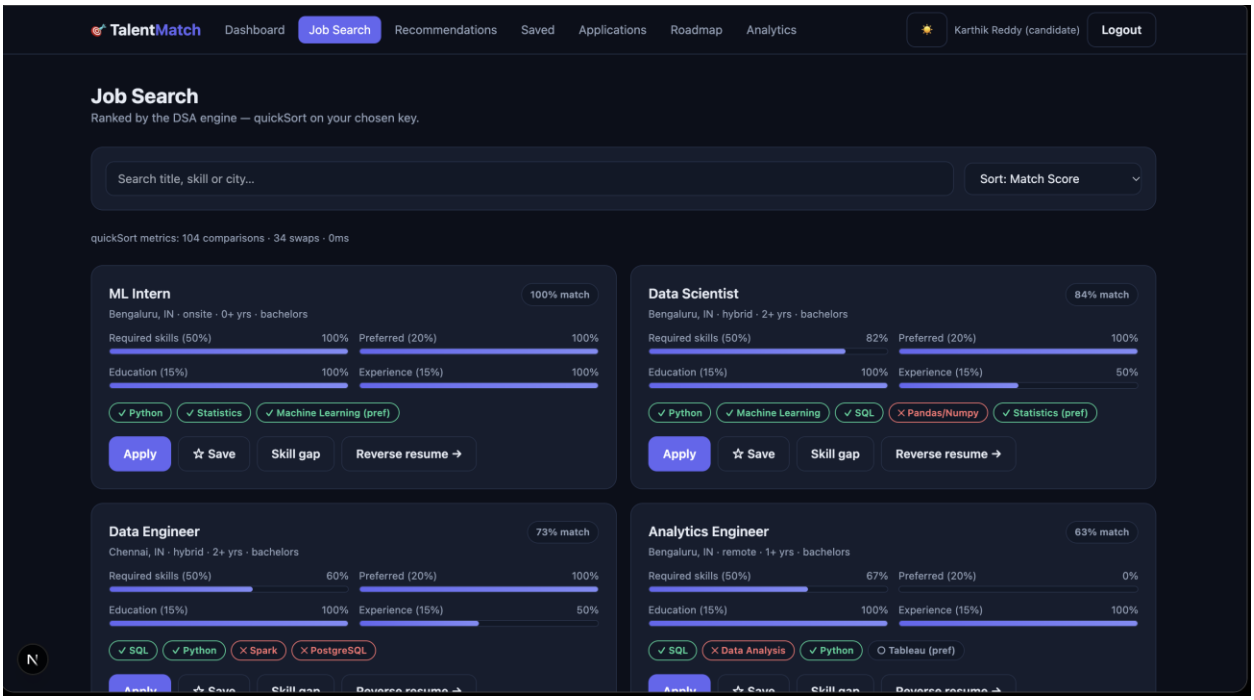
7.3 Candidate Dashboard

Description: The candidate dashboard presents profile information, skills, and access to job-search and recommendation functionality. When matching is enabled, the dashboard can help candidates explore opportunities based on the compatibility of their skills with job requirements.



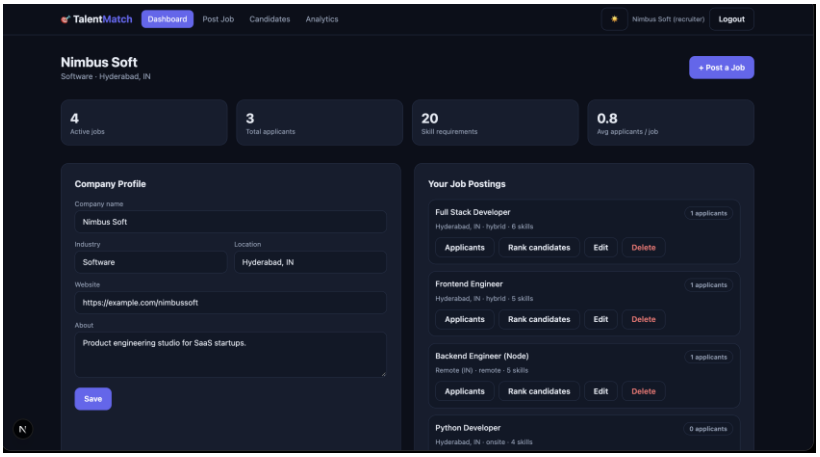
7.4 Job Search and Matching

Description: The job-search interface allows candidates to discover available positions using search terms and relevant filters. Matching scores can be used to order results according to skill compatibility, provided that the scoring logic is connected to the displayed results.



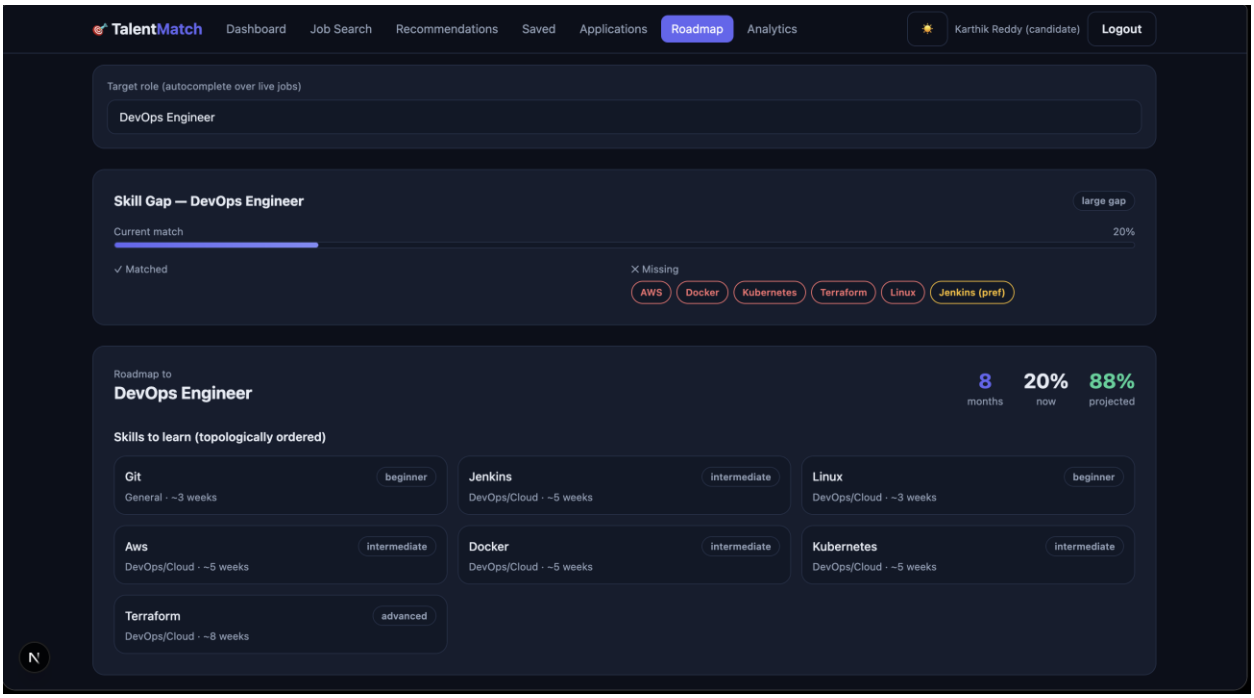
7.5 Recruiter Dashboard

Description: The recruiter dashboard provides access to job-posting and candidate-discovery features. Recruiters can review candidate profiles and, where implemented, compare ranked candidates according to their suitability for a selected role.



7.6 Reverse Resume and Skill-Gap Analysis

Description: The Reverse Resume component compares the skills associated with a candidate's current profile against the requirements of a target job. It identifies missing skills and can present a learning roadmap to help the candidate prepare for the desired role.



7.7 Backend and Database Verification

Insert Screenshot 7.7: Successful Backend or Database Test

Description: This test demonstrates whether the application backend responds correctly and whether the database connection can execute the required operations. The screenshot should show actual test output, such as a successful health-check response or a successful database read.

7.8 Discussion of Results

The system is designed to demonstrate how data structures and algorithms can support a recruitment-oriented application. The matching engine calculates compatibility between candidate skills and job requirements, while searching and sorting operations help organize the results. Trie-based autocomplete can reduce the effort needed to enter search terms, and priority queues can support the efficient retrieval of top recommendations.

The practical usefulness of these components depends on their integration with the user interface, backend services, and persistent database. Testing should confirm that the scores displayed to

users correspond to the actual data and that updates to profiles or job requirements are reflected in subsequent searches.

Test Case Summary

Complete this table after running the application.

Test case	Expected result	Actual result
Start the application	Local server starts without errors	[Pass/Fail]
Open the home page	Page loads correctly	[Pass/Fail]
Register a candidate	Valid account is created	[Pass/Fail]
Log in	Valid credentials allow access	[Pass/Fail]
Create a job	Job is stored and displayed	[Pass/Fail]
Search for a job	Relevant job results appear	[Pass/Fail]
Calculate skill match	Score reflects matching skills	[Pass/Fail]
Generate recommendations	Results are ordered as expected	[Pass/Fail]
Check database connectivity	Database operation succeeds	[Pass/Fail]
Generate a career roadmap	Missing skills or roadmap appear	[Pass/Fail]

Discussion: The final discussion should summarize the successful test cases, explain any failures or limitations, and identify which DSA components were verified. If some features remain incomplete, state this clearly rather than presenting them as successfully tested.

8. CONCLUSION & FUTURE SCOPE

The Résumé–Job Matching & Talent Marketplace Engine is designed to address the challenges associated with identifying suitable employment opportunities and matching candidates with recruiter requirements. The project applies concepts from Data Structures and Algorithms to a practical recruitment-oriented application.

The proposed system brings together candidate profiles, job listings, skill-based matching, recommendation ranking, and career-development guidance. Hashing supports efficient lookup, tries support prefix-based searching, sorting algorithms organize results, priority queues facilitate top-ranked recommendations, and graph representations can model relationships between candidates, skills, and jobs. The Reverse Resume component extends the system beyond conventional job discovery by identifying skill gaps between a candidate's current profile and a target role.

This project provides an opportunity to understand the practical relationship between algorithmic design and full-stack application development. It also highlights the importance of database integration, input validation, authentication, and testing when building a functional web application.

The overall effectiveness of the system depends on the correctness of its algorithms, the quality of the available candidate and job information, and the successful integration of its frontend, backend, and database components. The project establishes a foundation for further improvements in personalized job matching, recommendation quality, and career planning.

The project can be enhanced in the following stages.

A. Short-Term Enhancements

1. Improved résumé processing

Introduce automated PDF résumé parsing to extract candidate skills, educational qualifications, projects, and experience, reducing the need for manual data entry.

2. Advanced job filtering

Add filters for job location, experience level, employment type, salary range, and required skills to improve the relevance of search results.

3. Improved ranking explanations

Display the skills that contributed to a match score and explain why a job or candidate was ranked at a particular position.

4. Better input validation and error handling

Improve form validation, API error messages, and database-operation handling to make the application more reliable.

B. Medium-Term Enhancements

1. Machine-learning-based recommendations

Investigate machine-learning techniques that learn from job descriptions, candidate profiles, and user preferences to improve recommendations beyond fixed scoring rules.

2. Personalized learning roadmaps

Extend the Reverse Resume component to recommend learning resources and projects according to the candidate's missing skills and current proficiency.

3. Notifications and application tracking

Provide notifications for application updates, relevant job openings, and recruiter responses.

4. Recommendation evaluation

Evaluate recommendation quality using appropriate measures such as Precision@K, Recall@K, and user feedback, rather than relying only on matching scores.

C. Long-Term Enhancements

1. Labour-market skill analysis

Analyse job listings over time to identify commonly requested skills and changing industry requirements.

2. Explainable matching

Provide transparent explanations of recommendation scores so that candidates and recruiters can understand the factors influencing rankings.

3. Multilingual support

Extend the application to support additional languages and improve accessibility for users from different backgrounds.

4. Scalable architecture

Optimize database queries, caching, indexing, and recommendation processing to handle larger collections of candidates, skills, and job listings.

5. External platform integration

Explore integrations with professional networking platforms, learning-management systems, and course providers, subject to their APIs and access policies.

9. REFERENCES

1. LinkedIn, “LinkedIn Jobs.” Available at: https://www.linkedin.com/jobs/

2. Indeed, “Indeed Job Search.” Available at: https://www.indeed.com/

3. Naukri.com, “Job Search and Recruitment.” Available at: https://www.naukri.com/

4. Glassdoor, “Jobs and Company Information.” Available at: https://www.glassdoor.com/

5. Internshala, “Internships and Jobs.” Available at: https://internshala.com/

6. Monster, “Monster Jobs.” Available at: https://www.monster.com/

7. Wellfound, “Startup Jobs and Recruitment.” Available at: https://wellfound.com/

8. Google, “Google for Jobs Search.” Available at: https://www.google.com/search?q=jobs

9. Node.js, “Node.js Documentation.” Available at: https://nodejs.org/docs/latest/api/

10. Express.js, “Express Web Framework.” Available at: https://expressjs.com/

11. Supabase, “Supabase Documentation.” Available at: https://supabase.com/docs

12. Mozilla, “JavaScript Guide.” Available at: https://developer.mozilla.org/en-US/docs/Web/JavaScript/Guide

13. Git, “Git Documentation.” Available at: https://git-scm.com/doc

14. GitHub, “GitHub Documentation.” Available at: https://docs.github.com/

15. Cormen, T. H., Leiserson, C. E., Rivest, R. L., and Stein, C., Introduction to Algorithms, 4th ed., MIT Press, 2022.

16. Sedgewick, R., and Wayne, K., Algorithms, 4th ed., Addison-Wesley Professional, 2011.